

Dossier Technique & Manuel Utilisateur

Projet DevOps - Application GreenDesk

GreenDesk - Documentation officielle

Version strictement alignée sur le plan du document de référence, adaptée au contexte GreenDesk.

Version : v1.1.0

Date : 26 février 2026

Type : Dossier technique + manuel utilisateur

Auteurs / Équipe

- Hadi ISSA
- Fatima SAIDI
- Lydia AMROUCHE
- Misasoa ROBISON
- Mamadou DIALLO



 Repository GitHub

github.com/Misasoarobison/GreenDesk



 Documentation en ligne

misasoarobison.github.io/GreenDesk

Export PDF : le bouton *Exporter en PDF* en haut de page déclenche l'impression PDF (style A4 optimisé).

Sommaire

- **1. Présentation Générale**
 - 1.1 Objectif du Projet
 - 1.2 Équipe & Contributeurs
 - 1.3 Gestion de Projet & DevOps
- **2. Concurrence**
 - 2.1 Étude de la concurrence
 - 2.2 Utilisabilité & Design
- **3. Architecture Technique**
 - 3.1 Stack Technologique
 - 3.2 Modélisation (UML) & Structure des Données
- **4. Fonctionnalités Détaillées (User Guide)**
 - 4.1 Feature 1 - Gestion des espèces et plantes CRUD
 - 4.2 Feature 2 - Simulation évolutive d'une plante
 - 4.3 Feature 3 - Forêts & saisons
 - 4.4 Feature 4 - Effets & stimuli
 - 4.5 Feature 5 - Simulation & alertes
 - 4.6 Feature 6 - Simulation écosystème
 - 4.7 Feature 7 - Greenhouse Ops (KPI / ROI)
 - 4.8 Feature 8 - Capteurs (Sensor Readings)
- **5. Matrice de Responsabilités & Réalisations**
- **6. Tests effectués**
 - 6.1 Couverture
 - 6.2 CI/CD (Documentation complète + CI)
 - 6.3 Captures qualité
- **7. Guide d'Installation & Déploiement**

-
- **7.1 Prérequis**
 - **7.2 Exécution locale**
 - **7.3 Exécution Docker**
 - **7.4 Vérifications rapides**
 - **7.5 Dépannage**

- **8. Annexe API REST**

- **8.1 Base URL**
 - **8.2 Endpoints principaux**
 - **8.3 Exemples payload**
 - **8.4 Format d'erreur structuré**
-

1. Présentation Générale

1.1 Objectif du Projet

Le projet GreenDesk vise à centraliser les opérations clés de gestion agronomique dans une application unique. Le système permet :

- la gestion des espèces et des plantes,
- la simulation d'environnements (**forêts, saisons, effets, stimuli**),
- la supervision des alertes,
- l'analyse de KPI et d'indicateurs **ROI**.

L'objectif est de fournir une base décisionnelle fiable, testée et documentée, utilisable autant par les opérateurs métiers que par l'équipe technique.

Qui ?

- **Opérateurs serre** : utilisent les fonctions terrain (espèces, plantes, forêts, interventions).
- **Responsables agronomiques** : pilotent les alertes, la simulation et les décisions **KPI/ROI**.
- **Équipe technique** : maintient l'**API**, la qualité logicielle et l'exploitation.

Quoi ?

- Une plateforme unique pour gérer les données agronomiques, simuler les évolutions et suivre la performance.
- Un socle **API** documenté et testable pour intégrer des interfaces et automatisations.

Pourquoi ?

- Réduire la dispersion des informations et améliorer la traçabilité opérationnelle.
- Accélérer la prise de décision grâce à des indicateurs consolidés.
- Limiter les régressions via une approche qualité/CI continue.

Scénario d'usage (application actuelle)

Objectif

Décrire le flux opérationnel réel de l'application GreenDesk, de la création de plante jusqu'au pilotage par indicateurs.

Quoi ?

Un scénario métier concret qui combine création, simulation, alertes, corrections et suivi KPI dans le fonctionnement actuel.

Contexte : exploitation quotidienne d'une serre simulée dans GreenDesk.

Acteurs

- Opérateur serre
- Responsable agronomique
- Manager exploitation

Déroulé du scénario

1. L'opérateur crée une espèce puis une plante (`/api/species` , `/api/plants/create`).
2. Il crée une forêt et positionne la plante dans la grille (`/api/forests` , `/api/forests/{forestId}/plants`).
3. Le responsable lance une simulation (`/api/ecosystem/tick` ou `/api/ecosystem/simulate/{n}`) pour observer l'évolution.
4. L'équipe consulte l'état et les alertes de la plante (`/status` , `/plants/{plantId}/alerts`).
5. Si besoin, elle applique des effets/stimuli correctifs (`/api/plants/{plantId}/effects/{effectId}` , `/api/stimuli`).
6. Les alertes traitées sont acquittées (`/alerts/{alertId}/ack`) et un nouveau cycle est relancé.
7. Le manager vérifie les KPI opérationnels (`/api/greenhouse/overview` , `/api/greenhouse/roi`) pour décider des actions suivantes.

Résultat attendu

- Cycle complet de gestion plante -> forêt -> simulation -> correction.
- Diminution du stress/alertes après intervention.
- Décision pilotée par indicateurs **Greenhouse Ops**.

1.2 Équipe & Contributeurs

Membres du groupe :

- Hadi ISSA
- Fatima SAIDI
- Lydia AMROUCHE
- Misasoa ROBISON
- Mamadou DIALLO

Répartition des contributions :

- Produit & cadrage : vision, backlog, priorités métier.
- Backend & API : contrôleurs, services, persistance.
- Qualité & CI : tests, couverture, validation pipeline.
- Documentation : dossier unique, annexes, export PDF.

Répartition par langage et par membre

Langage / Technologie	Hadi ISSA	Fatima SAIDI	Lydia AMROUCHE	Misaso ROBISON	Mamadou DIALLO
Java	3	2	3	4	3
Spring Boot	3	2	3	4	3
MongoDB	2	1	2	4	2
JUnit / Tests	4	2	2	2	2
Gradle / Build	3	3	2	2	1
HTML / CSS / JS	1	1	3	2	1
Markdown / Docs	2	2	4	1	1
Mermaid / UML	2	1	4	2	1
Git / CI-CD	3	3	2	2	2
Architecture	3	2	3	3	3

Légende : 1 = Faible | 2 = Moyen | 3 = Fort | 4 = Très fort

1.3 Gestion de Projet & DevOps

L'organisation suit une approche itérative inspirée Agile/Scrum :

- cycles courts orientés valeur,
- revues PR systématiques,
- intégration continue orientée qualité,
- documentation maintenue dans le même cycle que le code.

Mécanismes opérationnels

- Versioning **Git** avec branches feature.
- **Contrôle qualité local** : `./gradlew clean check`.
- **Validation CI** : tests + JaCoCo + cohérence doc.
- **Traçabilité** : PR explicites avec impact API/métier.

2. Concurrency

2.1 Étude de la concurrence

L'analyse concurrentielle est réalisée sur trois familles de solutions utilisées dans des contextes proches de GreenDesk.

Type d'outil	Forces	Limites	Écart couvert par GreenDesk
Tableurs / scripts	Démarrage très rapide, peu de friction	Données dispersées, faible traçabilité, logique difficile à maintenir	Référentiel centralisé + API versionnable + règles métier explicites
Plateformes IoT orientées capteurs	Excellente télémétrie temps réel	Faible profondeur métier agronomique, faible simulation native	Intègre capteurs + états plante + alertes + simulation dans un même modèle
Simulateurs spécialisés	Moteurs avancés de simulation	Coûts/licences élevés, intégration SI plus lourde	Approche pragmatique API-first, plus légère à intégrer et exploiter

Lecture stratégique

- Les outils simples sont rapides, mais deviennent coûteux en maintenance quand le périmètre grandit.
- Les outils très spécialisés sont puissants, mais peuvent être surdimensionnés pour un usage opérationnel quotidien.
- **GreenDesk** cible une zone d'équilibre : suffisamment structuré pour durer, suffisamment simple pour rester exploitable.

Positionnement GreenDesk

- Architecture API-first pour faciliter intégration et automatisation.
- Couplage métier + qualité (tests, couverture, CI) pour fiabilité continue.
- Vision opérationnelle complète : espèces → plantes → forêts → alertes/simulation → KPI/ROI.

2.2 Utilisabilité & Design

L'utilisabilité est pensée pour réduire le temps entre observation et action.

Principes UX retenus

-
- Navigation orientée tâches métier (créer, diagnostiquer, corriger, vérifier).
 - Accès rapide aux points critiques (état plante, alertes, KPI, ROI).
 - Lisibilité forte des signaux (stress, sévérité, tendances).
 - Documentation mono-page pour lecture continue et export PDF instantané.

Décisions de design

- Interface responsive pour usage sur postes variés.
- Hiérarchie visuelle stable (titres, blocs, tableaux, callouts).
- Cohérence de vocabulaire entre UI, API et documentation.
- Diagrammes Mermaid + captures réelles pour accélérer la compréhension.

Critères d'utilisabilité visés

- Comprendre les informations clés en moins de 30 secondes sur un écran de synthèse.
- Retrouver une action principale en moins de 3 clics.
- Identifier un état anormal sans ambiguïté grâce aux alertes et statuts.

Bénéfice attendu

- Moins d'erreurs d'interprétation.
 - Décisions plus rapides côté exploitation.
 - Meilleure adoption par les profils non techniques.
-

3. Architecture Technique

3.1 Stack Technologique

Backend

- **Langage** : Java 21
- **Framework** : Spring Boot 3.3.3
- **Architecture** : MVC/REST (**Controller** -> **Service** -> **Repository**)
- **Persistence** : MongoDB (Spring Data)

Frontend / Documentation

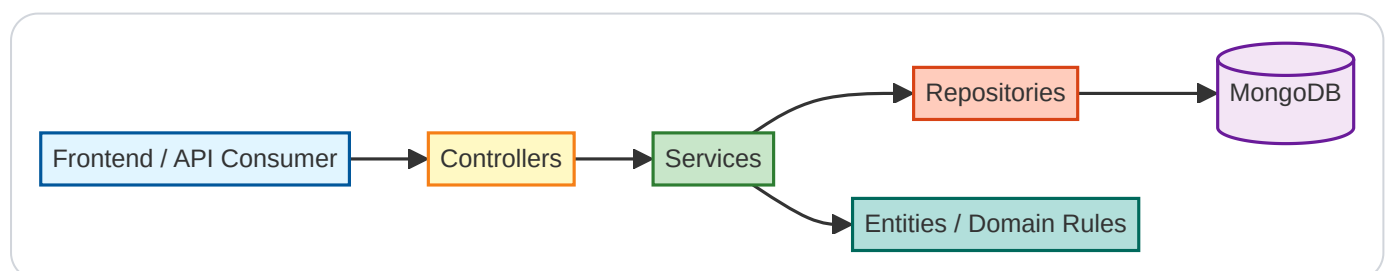
- **UI app** : HTML5/CSS3/JavaScript
- **Documentation** : Docsify + Mermaid

Build & Qualité

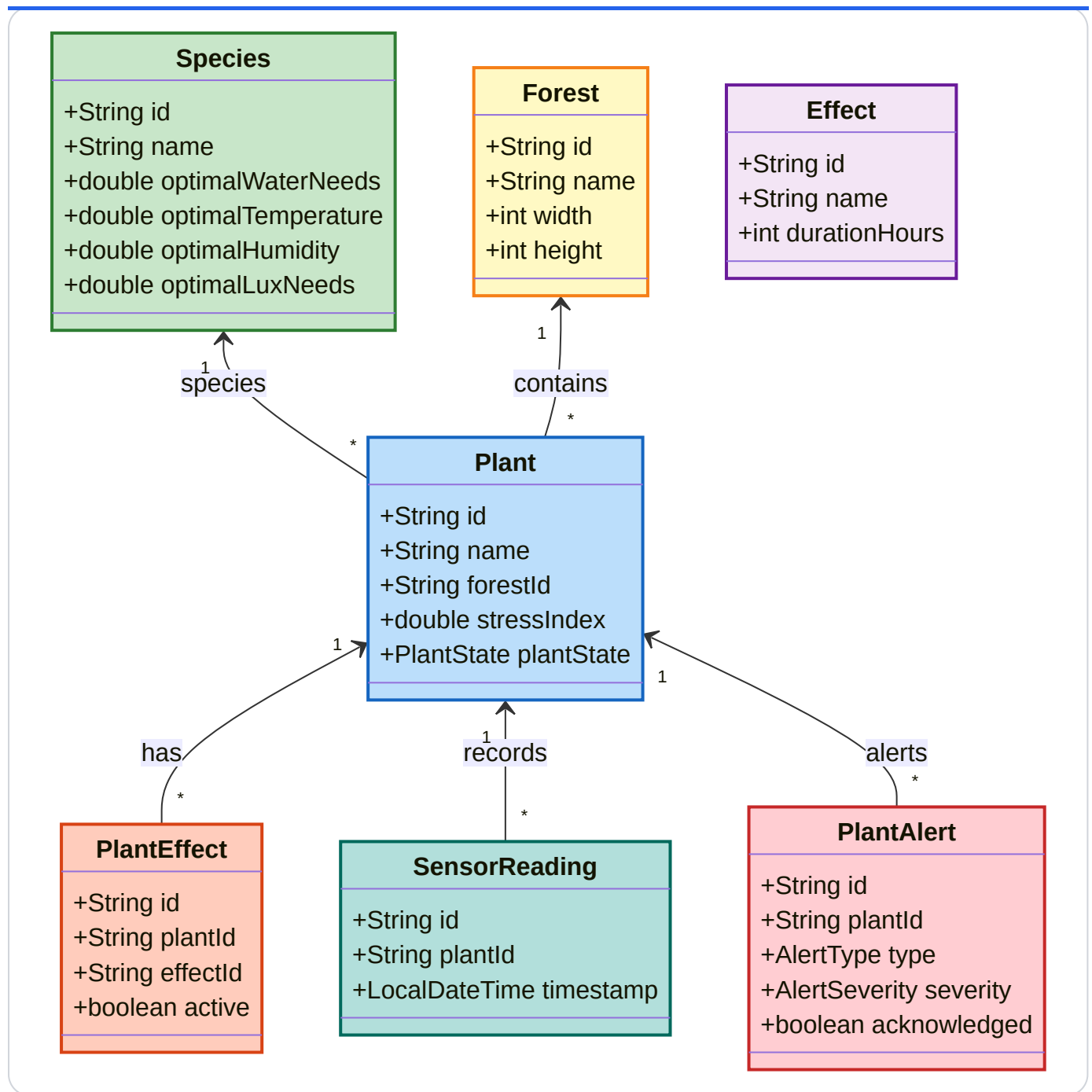
- **Build** : Gradle Wrapper
- **Tests** : JUnit + MockMvc
- **Couverture** : JaCoCo
- **API interactive** : Swagger/OpenAPI

3.2 Modélisation (UML) & Structure des Données

3.2.1 Diagramme d'architecture

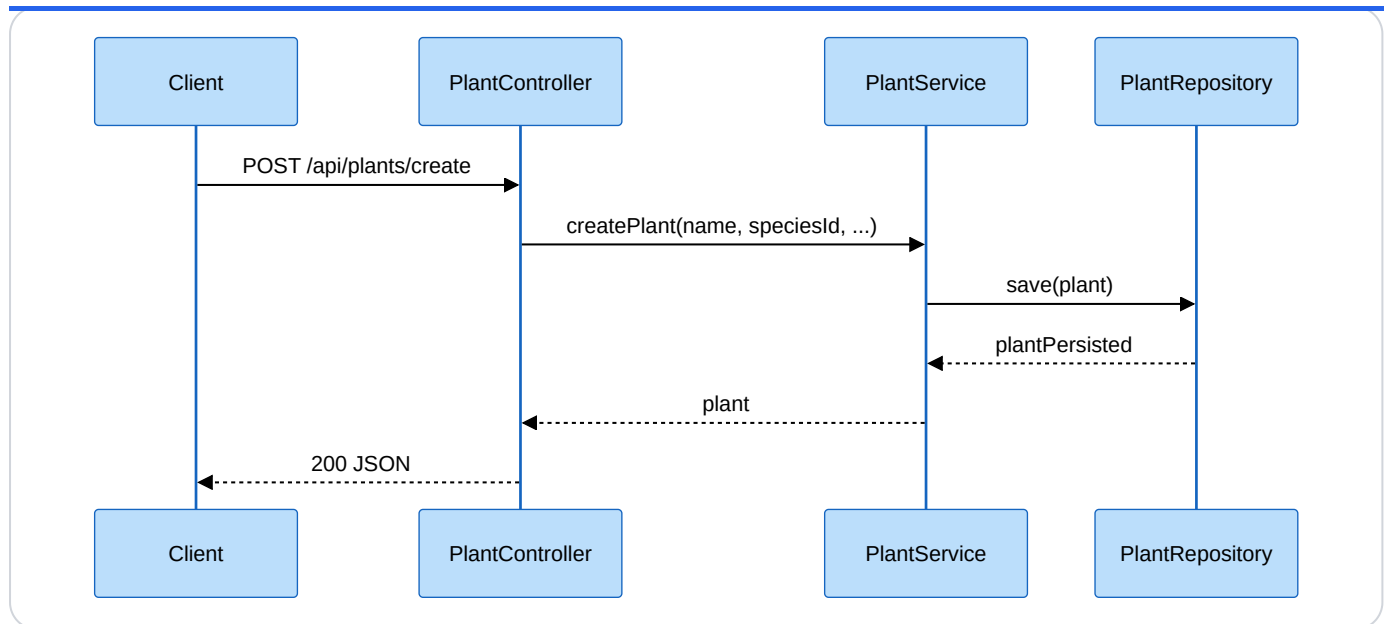


3.2.2 Diagramme de classes (back)

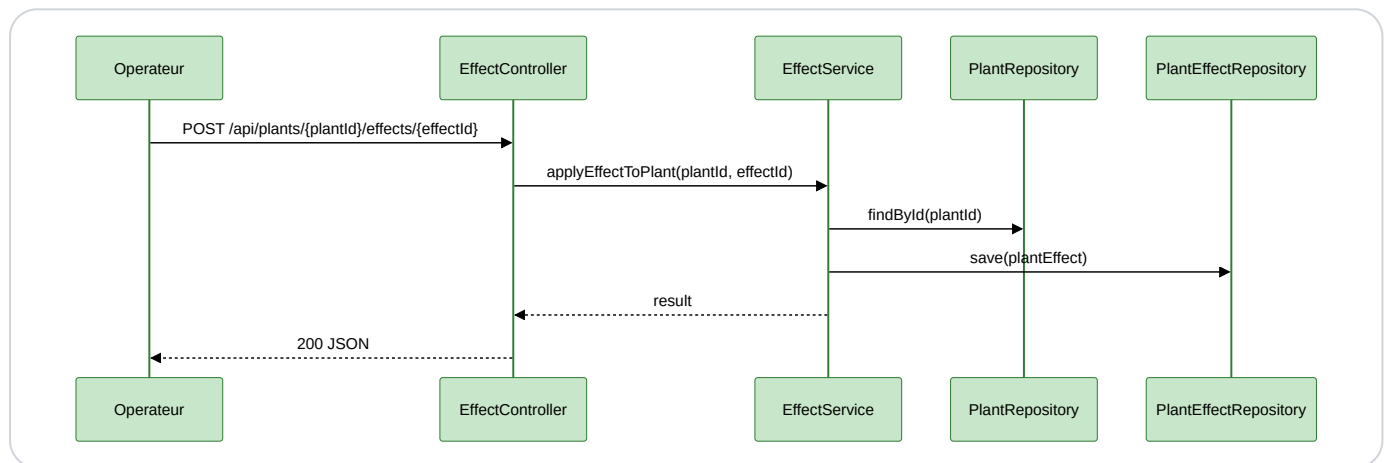


3.2.3 Diagrammes de séquence (back)

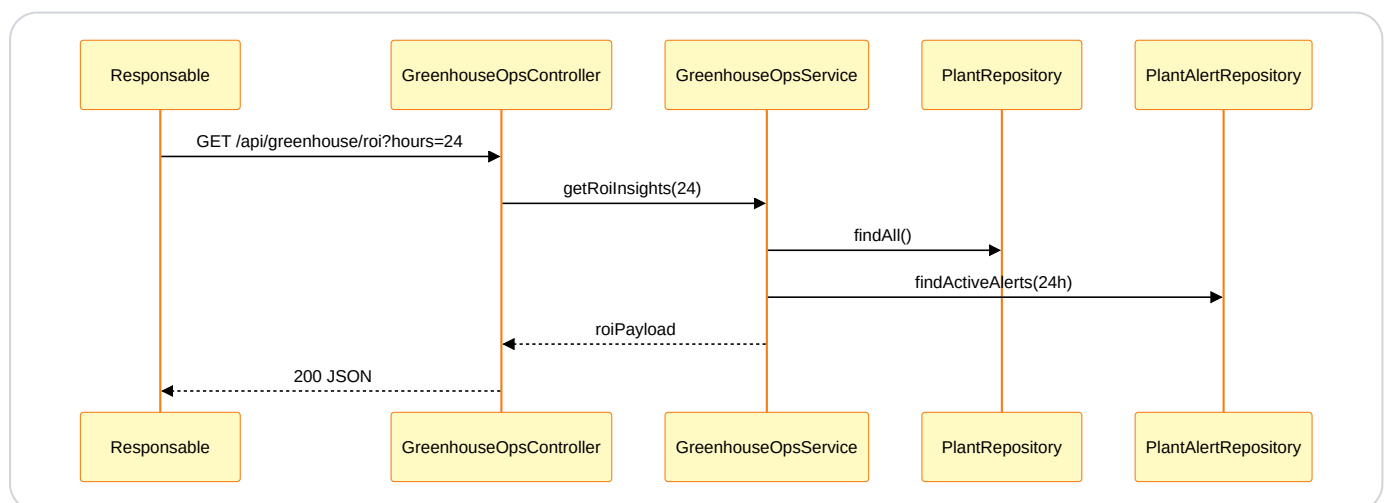
Création d'une plante



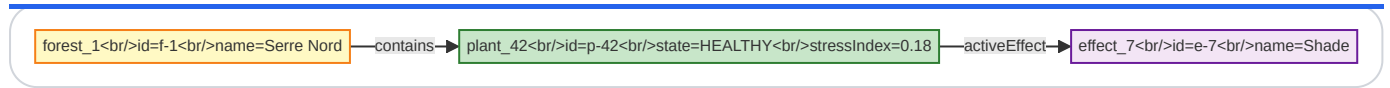
Application d'un effet



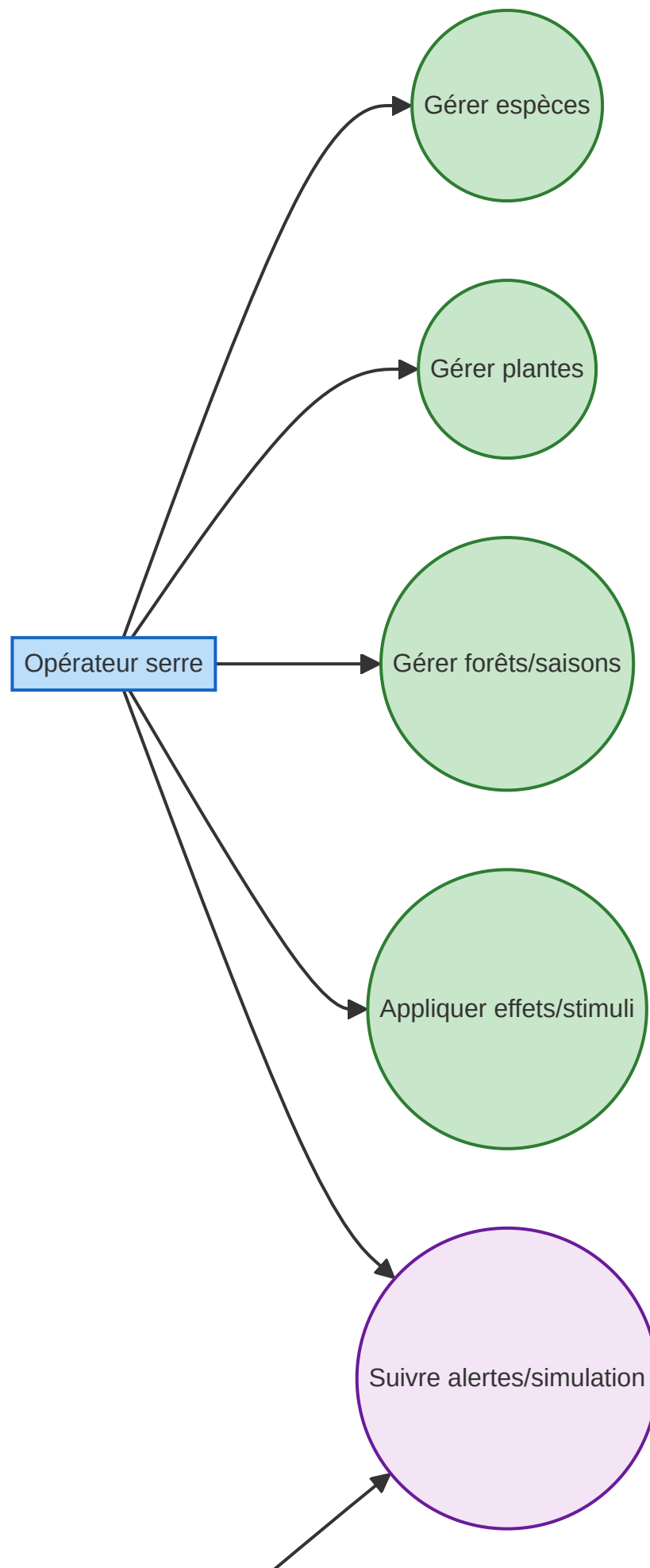
Consultation ROI

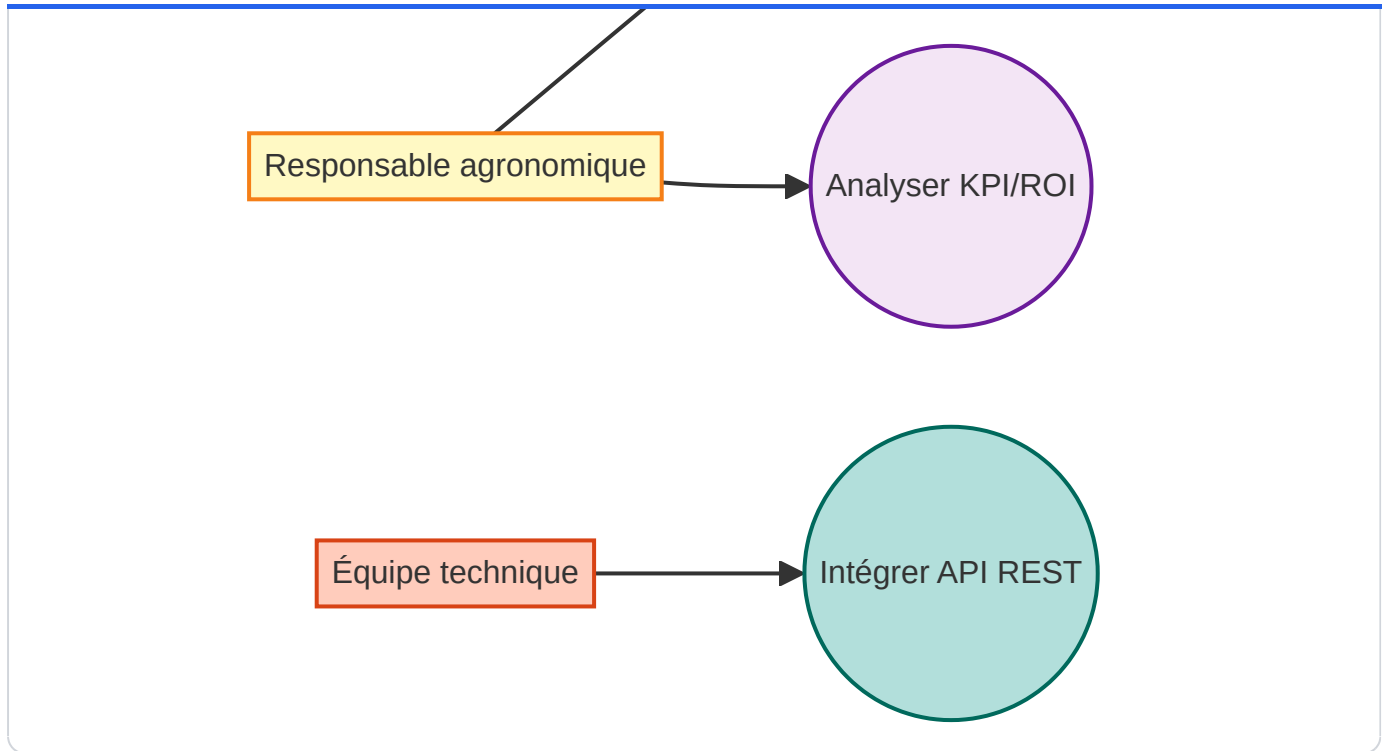


3.2.4 Diagramme d'objet (back)

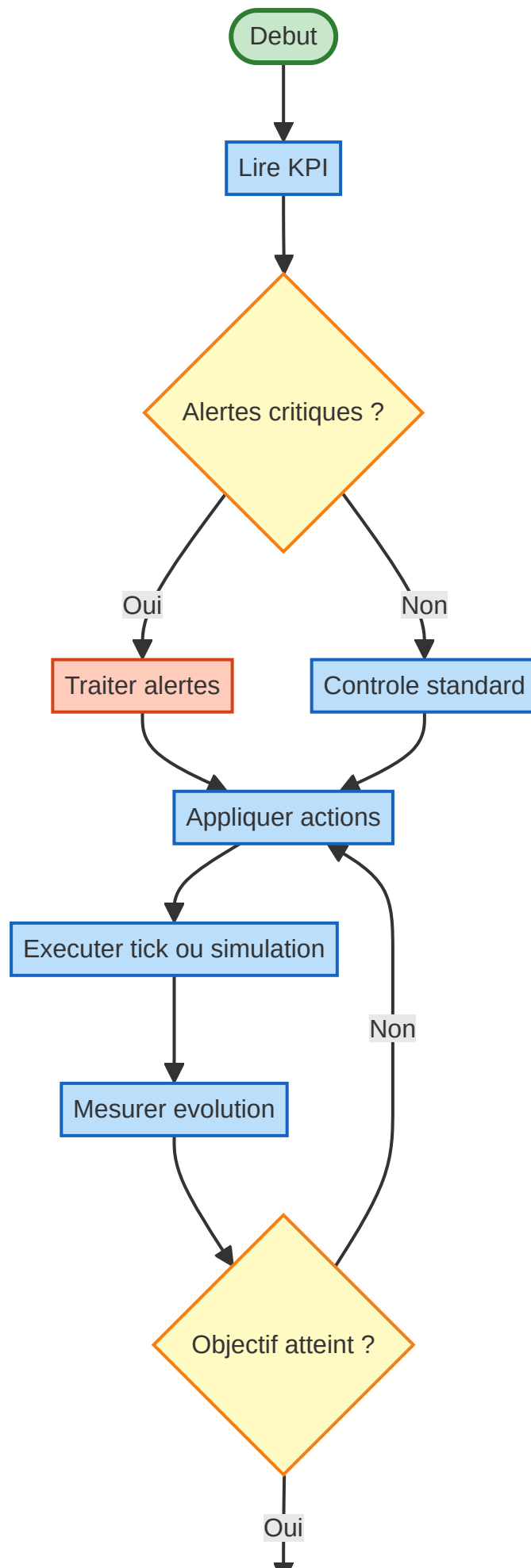


3.2.5 Diagramme de cas d'utilisation



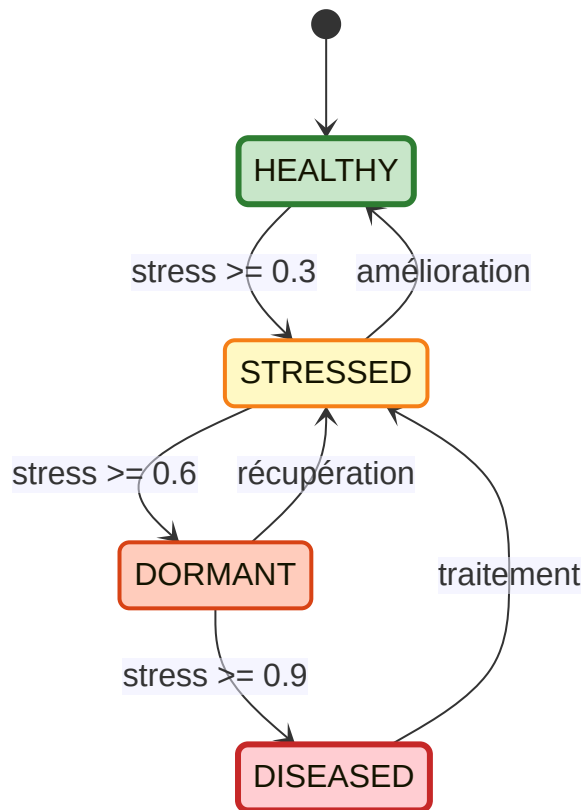


3.2.6 Diagramme d'activité



Fin

3.2.7 Diagramme d'état



4. Fonctionnalités Détaillées (User Guide)

Exigence stricte : 6 fonctionnalités, chacune avec But feature, Scénarios/Personas, Wireframes/Screenshots, Résumé NVF.

4.1 Feature 1 - Gestion des espèces et plantes CRUD

But feature : centraliser le référentiel agronomique.

Scénarios / Personas

- Persona : Opérateur serre
- Scénario : création d'une espèce puis réutilisation lors de la création de plantes.

Wireframe / screenshot

The screenshot displays the 'GreenDesk' application interface. At the top, a green navigation bar contains the logo and several menu items: 'Parcours', 'Vue d'ensemble', 'Simulation', 'Especies' (highlighted), 'Effets', and 'Maladies'. Below the navigation bar, the main content area is titled 'ETAPE 1 - PREPARER' and 'Configurer les espèces de référence'. A subtitle reads: 'Chaque espèce représente un comportement cible. Une bonne configuration améliore toute la suite du parcours utilisateur.' Below this, there are two side-by-side boxes. The left box, 'Bonnes pratiques de paramétrage', lists three points: 'Créer au moins 3 espèces contrastées (eau/lumière/température).', 'Utiliser des noms explicites pour faciliter la comparaison inter-scénarios.', and 'Vérifier le catalogue avant de planter dans la simulation.' The right box, 'ESPÈCES AFFICHÉES', shows a large number '4', 'Alertes actives serre: 0', 'Effets actifs en cours: 0', and a button '→ Aller en simulation'. Below these boxes is a 'Catalogue espèces' section with a search bar and a button '+ Nouvelle Espèce'. The search bar contains the text 'Filtrer par nom d'espèce...'. Below the search bar is a table with the following data:

NOM	BESOIN EAU (ML)	BESOIN LUM (LUX)	TEMP. IDÉALE (°C)	CROISSANCE DE BASE	ACTIONS
Oak	240 ml	3800	19°C	x0.42	
Pine	170 ml	3200	17°C	x0.48	
Tomate	220 ml	3000	24°C	x1.5	

Résumé NVF

- N : nécessaire pour définir les seuils de référence.
- V : valeur forte sur la cohérence des diagnostics.
- F : faisable via endpoints CRUD déjà exposés.

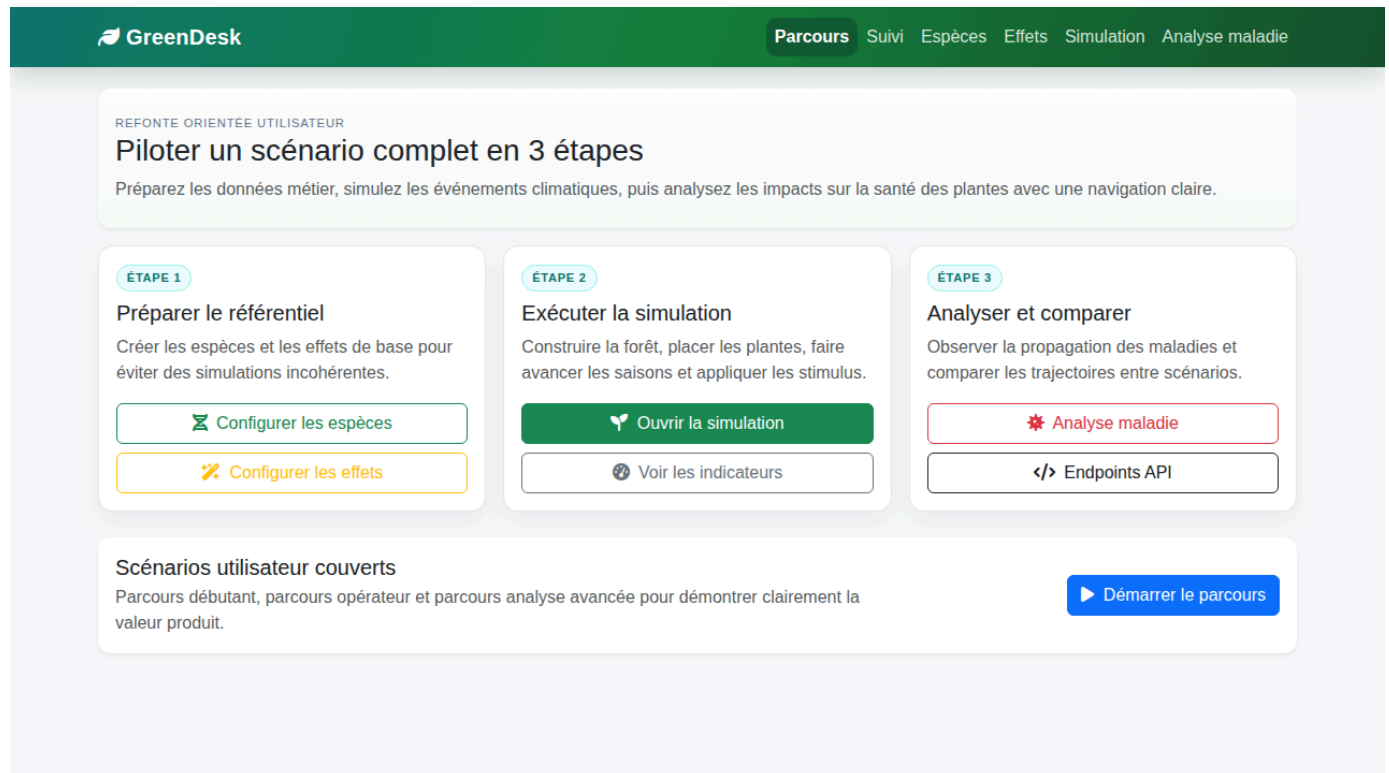
4.2 Feature 2 - Simulation évolutive d'une plante

But feature : suivre les plantes à granularité individuelle.

Scénarios / Personas

- Persona : Opérateur serre
- Scénario : création, lecture état/statut, comparaison de deux plantes.

Wireframe / screenshot



Résumé NVF

- N : indispensable pour le pilotage opérationnel.
- V : visibilité sur stress/état par individu.
- F : endpoints disponibles + logique métier stable.

4.3 Feature 3 - Forêts & saisons

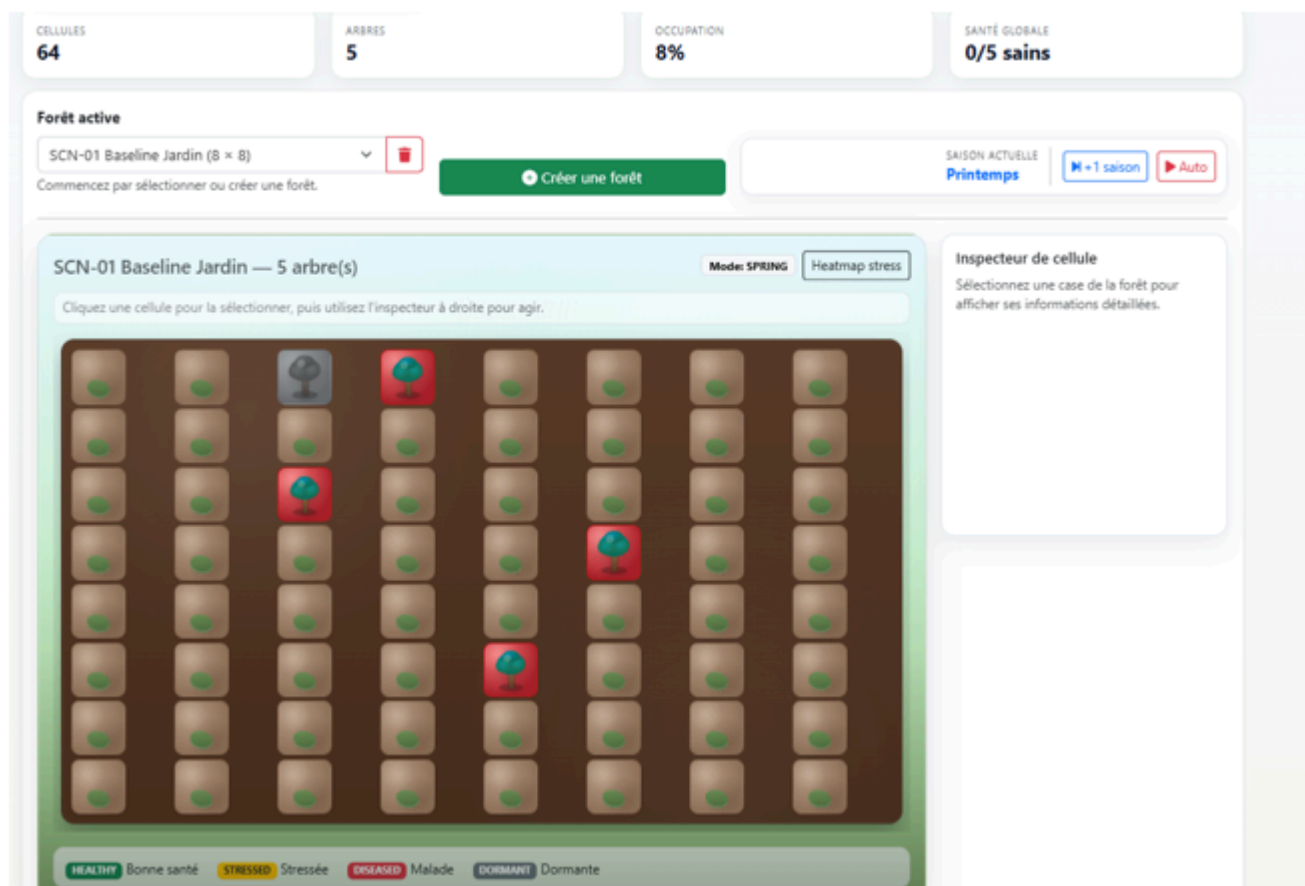
But feature : organiser la plantation dans l'espace et le temps.

Scénarios / Personas

- Persona : Responsable agronomique
- Scénario : créer forêt, placer plantes sans conflit, avancer cycle saisonnier.

Wireframe / screenshot

Capture - Forêt & saisons (test terrain)



Résumé NVF

- N : nécessaire à la simulation réaliste.
- V : améliore la planification des interventions.
- F : mécanismes de grille et season cycle implémentés.

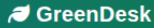
4.4 Feature 4 - Effets & stimuli

But feature : agir sur l'environnement simulé et observer les impacts.

Scénarios / Personnas

- Persona : Opérateur serre
- Scénario : appliquer un effet, déclencher un stimulus, vérifier évolution.

Wireframe / screenshot

ParcoursVue d'ensembleSimulationEspèces**Effets**Maladies

ETAPE 1 - PREPARER

Bibliothèque des effets de simulation

Construisez des interventions réutilisables pour accélérer les parcours "tester puis comparer".

Comment penser les effets pour l'utilisateur ?

- Créer des effets simples d'abord (eau, chaleur, croissance).
- Nommer selon l'intention métier: "Stress thermique court", "Soutien hydrique".
- Réutiliser ces effets dans la simulation pour garder un scénario lisible.

EFFETS VISIBLES
16
Alertes actives serre: 0
Lectures capteurs 24h: 0
[→ Utiliser en simulation](#)

Catalogue d'effets

Filtrez système/custom puis ajustez rapidement vos paramètres d'expérience.

Tous les effetsAffichés: 16

Extra Watering

Système

Arrosage supplémentaire

Durée: 4h

Stress reduction: 0.2

Humidité: 10

Eau: 50

Fertilizer

Système

Augmente la croissance

Durée: 12h

Growth: 0.5

Stress reduction: 0.15

Eau: 20

Heating

Système

Augmente la température ambiante

Durée: 8h

Stress reduction: 0.1

Temp: 5

Résumé NVF

- N : nécessaire pour passer de l'observation à l'action.
- V : accélère l'optimisation des conditions culturales.
- F : services et endpoints dédiés déjà présents.

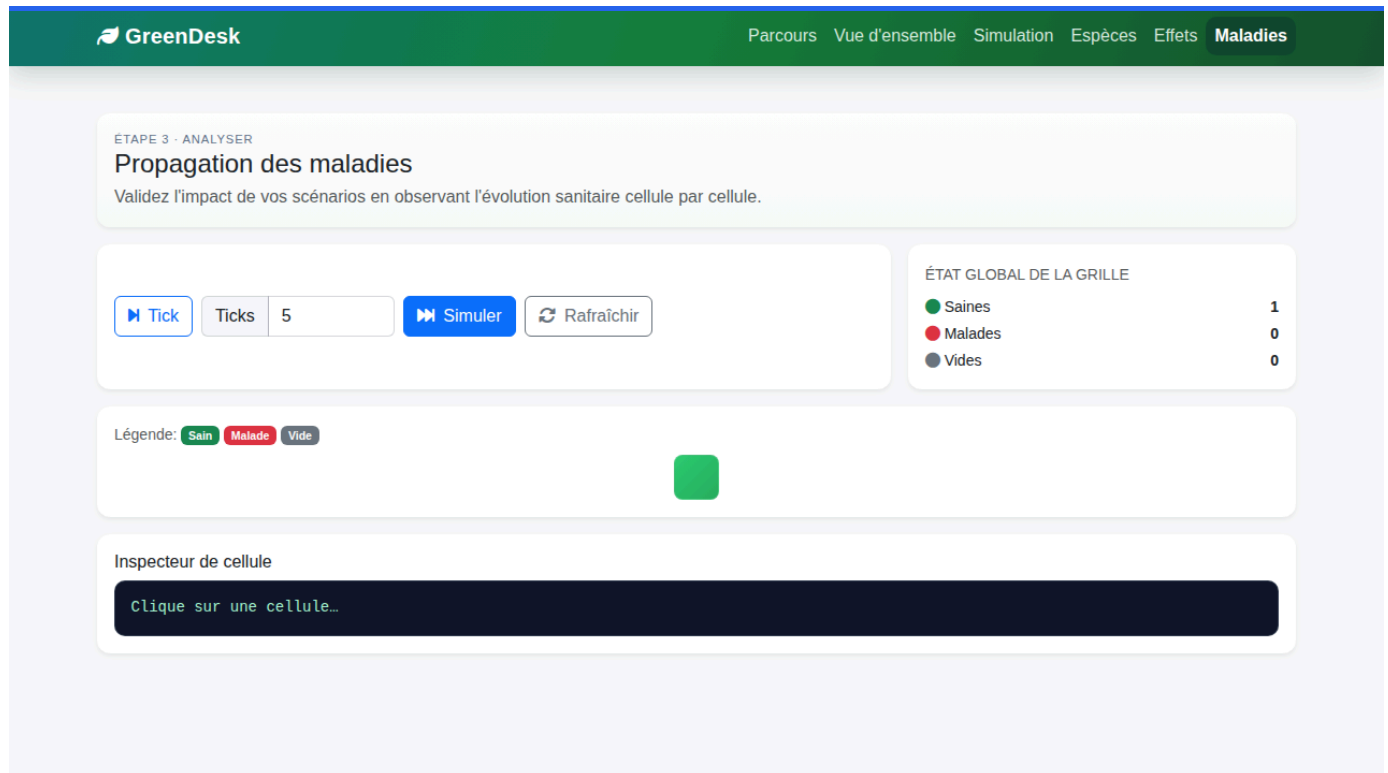
4.5 Feature 5 - Simulation & alertes

But feature : anticiper les dérives et gérer les incidents.

Scénarios / Personas

- Persona : Responsable agronomique
- Scénario : simuler plusieurs ticks, analyser alertes, acquitter les alertes traitées.

Wireframe / screenshot



Résumé NVF

- N : indispensable pour réduction du risque.
- V : priorisation par sévérité.
- F : module simulation + module alertes testés.

4.6 Feature 6 - Simulation écosystème

But feature : piloter une simulation globale multi-plantes/multi-forêts.

Scénarios / Personas

- Persona : Responsable agronomique / opérateur simulation
- Scénario : lancer `tick` , `simulate/{n}` ou `simulate/{forestId}/{n}` , puis analyser l'état des cellules (`cells`).

Wireframe / screenshot

Résumé NVF

- N : nécessaire pour simuler l'évolution à l'échelle système.
- V : permet d'anticiper les dérives et d'ajuster la stratégie.
- F : endpoints `EcosystemController` disponibles et testables.

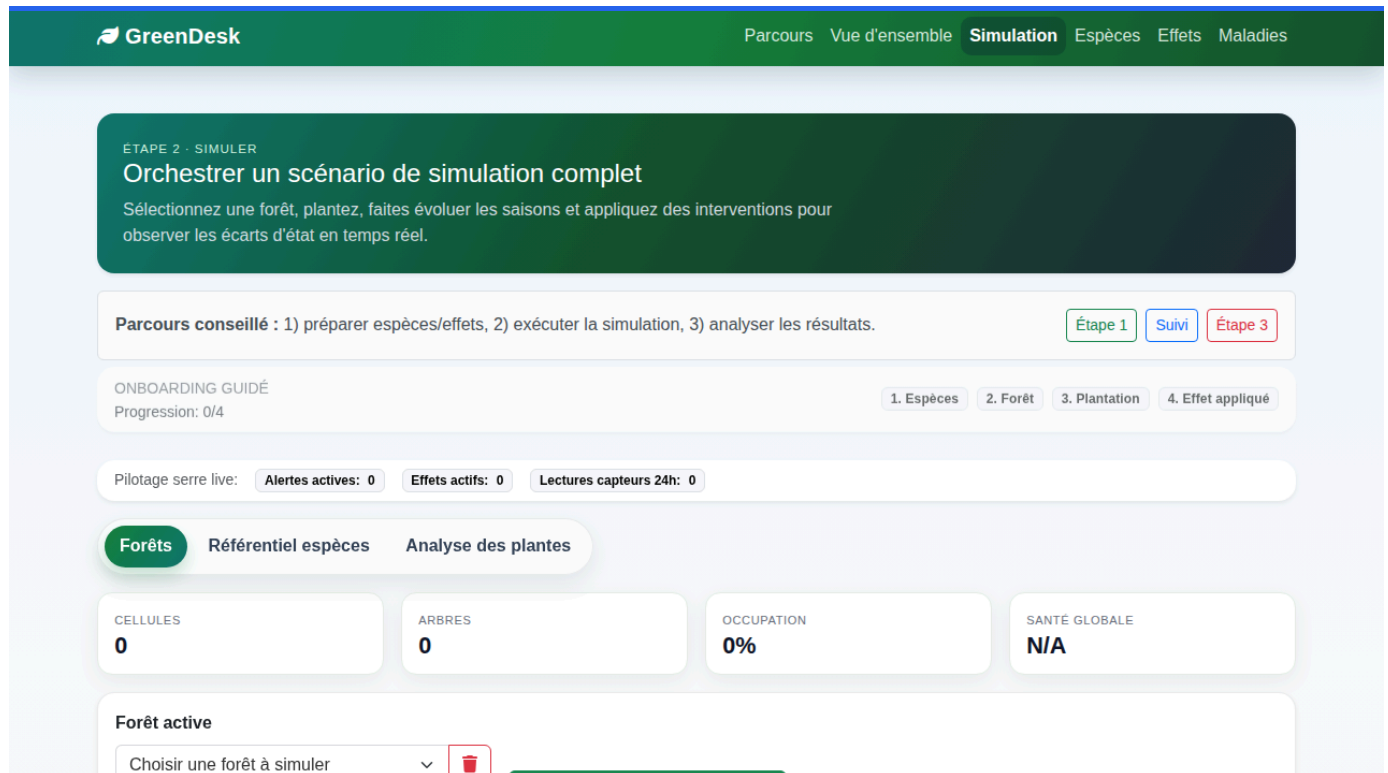
4.7 Feature 7 - Greenhouse Ops (KPI / ROI)

But feature : fournir des indicateurs décisionnels consolidés pour le pilotage.

Scénarios / Personas

- Persona : Responsable exploitation / Tech lead
- Scénario : consulter `overview` , `alerts` , `roi` , `roi/forests` puis déclencher `sensor-stream/tick` .

Wireframe / screenshot



Résumé NVF

- N : nécessaire pour le pilotage data-driven.
- V : améliore l'arbitrage coût/risque/performance.
- F : endpoints `GreenhouseOpsController` opérationnels.

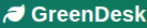
4.8 Feature 8 - Capteurs (Sensor Readings)

But feature : historiser et exploiter les mesures capteurs des plantes.

Scénarios / Personas

- Persona : Opérateur technique
- Scénario : injecter une mesure capteur, puis lire la dernière valeur (`latest`) pour une plante.

Wireframe / screenshot

 Vue d'ensemble Simulation Espèces Effets Maladies

SUIVI OPÉRATIONNEL

Cockpit de scénario

Suivez l'état global avant et après vos actions de simulation pour raconter un parcours utilisateur clair.

Vue de pilotage

Chargement...

Ce tableau de bord répond à la question: "où en est mon scénario en ce moment ?".

Fil conducteur : préparer (espèces/effets) → exécuter (simulation/saisons) → analyser (stress, maladies, comparaison).

Simuler

Espèces

Effets

Maladies

Espèces

0

Plantes

0

Forêts

0

Stress moyen

0%

Effets actifs

0

Alertes actives

0

Lectures 24h: 0

État des forêts

Action rapide: stimulus

Résumé NVF

- N : nécessaire pour relier simulation et données observées.
- V : améliore la détection précoce des anomalies.
- F : `SensorReadingController` et service dédiés implémentés.

5. Matrice de Responsabilités & Réalisations

Fonctionnalité / Axe	Lydia	Fatima	Hadl	Misaoa	Mamadou
Architecture Backend (Java 21 / Spring Boot)	x	x	x	x	x
Gestion BDD (MongoDB / Atlas)				x	
Gestion des Releases & CI/CD (GitHub Actions / Docker)			x		
Documentation & UML (Diagrammes de classes/séquence)	x	x			
Organisation & Pilotage Agile (Scrum / Backlog)	x	x	x	x	x
Gestion des Espèces (CRUD & Besoins optimaux)					
Gestion des Plantes (Calcul d'état & Stress Index)				x	x
Gestion des Forêts (Grilles & Positions x,y)	x				
Système de Saisons (Modificateurs environnementaux)	x				
Système d'Effets (Fertilizer, Shade, Heating, etc.)	x				
Simulation d'Écosystème (Propagation de maladies)			x	x	
Analyse comparative (Protocole de clonage & Stimuli)			x	x	
Plant Placement Optimizer		x			
Interface Web / Dashboard (Cockpit de pilotage)		x	x		
Rapports de Diagnostic (/status & capteurs)	x	x			
Supervision des Alertes (KPI & ROI)	x	x	x		
Tests Unitaires & Intégration (JUnit / Mockito)	x		x		

6. Tests effectués

6.1 Couverture

6.1.1 Taux de couverture

Indicateur	Taux actuel	Seuil cible
LINE	81.04%	>= 70%
BRANCH	52.47%	>= 45%
CLASS	98.18%	>= 90%

6.1.2 Tableau des tests effectués (54 suites)

Suites de tests exécutées	Type	Description	Objectif
Contrôleurs API (12) PlantAlertControllerTest EcosystemControllerTest EffectControllerTest SeasonControllerTest ForestControllerTest ForestSeasonControllerTest GreenhouseOpsControllerTest HomeControllerTest PlantControllerTest SensorReadingControllerTest SpeciesControllerTest StimulusControllerTest	Intégration API (MockMvc)	Vérifie les routes REST, codes HTTP, payloads, validations et gestion d'erreurs côté contrôleurs.	Garantir que les endpoints exposés sont conformes au contrat API et stables en régression.
Services métier (9) EcosystemServiceTest EffectServiceTest ForestServiceTest GreenhouseOpsServiceTest NotFoundTests PlantAlertServiceTest SeasonServiceTest SensorReadingServiceTest StimulusServiceTest	Unitaire métier	Teste les règles de gestion, calculs, transitions d'état et scénarios d'exception dans la couche service.	Valider la logique fonctionnelle centrale indépendamment du transport HTTP.
Repositories (8) EffectRepositoryTest ForestRepositoryTest PlantEffectRepositoryTest PlantRepositoryTest SeasonCycleRepositoryTest SensorReadingRepositoryTest SpeciesRepositoryTest StimulusRepositoryTest	Intégration persistance	Contrôle les opérations de lecture/écriture, requêtes et mapping avec la couche de persistance.	Sécuriser l'accès aux données et éviter les régressions de stockage/recherche.
Entités & modèle domaine (17) PlantAlertTest DiseasesTest EcosystemCellTest EcosystemTest EnvironmentDataTest SeasonCycleTest SeasonTest SeasonTypeTest ForestTest InterventionTest PlantEffectTest PlantStateTest PlantTest SensorReadingTest SpeciesTest StimulusTest TestEffects	Unitaire modèle	Vérifie invariants, comportements, transitions internes et cohérence des objets métier.	Assurer la robustesse du modèle de données utilisé par les services et simulations.

Suites de tests exécutées	Type	Description	Objectif
Scénarios transverses / legacy (8) <code>TestEcosystemServices</code> <code>TestEffects</code> <code>TestForestAndSeasons</code> <code>TestPlantLifecycle</code> <code>TestPlantServices</code> <code>TestSensorReadingsAndAlerts</code> <code>TestSimulationEnvironment</code> <code>TestSpeciesServices</code>	Intégration fonctionnelle	Exécute des scénarios bout-en-bout couvrant plusieurs composants en chaîne (services + domaine + API).	Valider les parcours métier complets et la cohérence globale de l'application.

6.1.3 Outils utilisés

- **JUnit 5** : exécution des tests unitaires et d'intégration.
- **Spring MockMvc** : tests contrôleurs/API.
- **JaCoCo** : mesure de couverture (LINE/BRANCH/CLASS).
- **Gradle Wrapper** : orchestration build + tests + rapport.

Commandes principales :

```
./gradlew test
./gradlew test jacocoTestReport
./gradlew clean check
```

bash

6.1.4 Capture JaCoCo

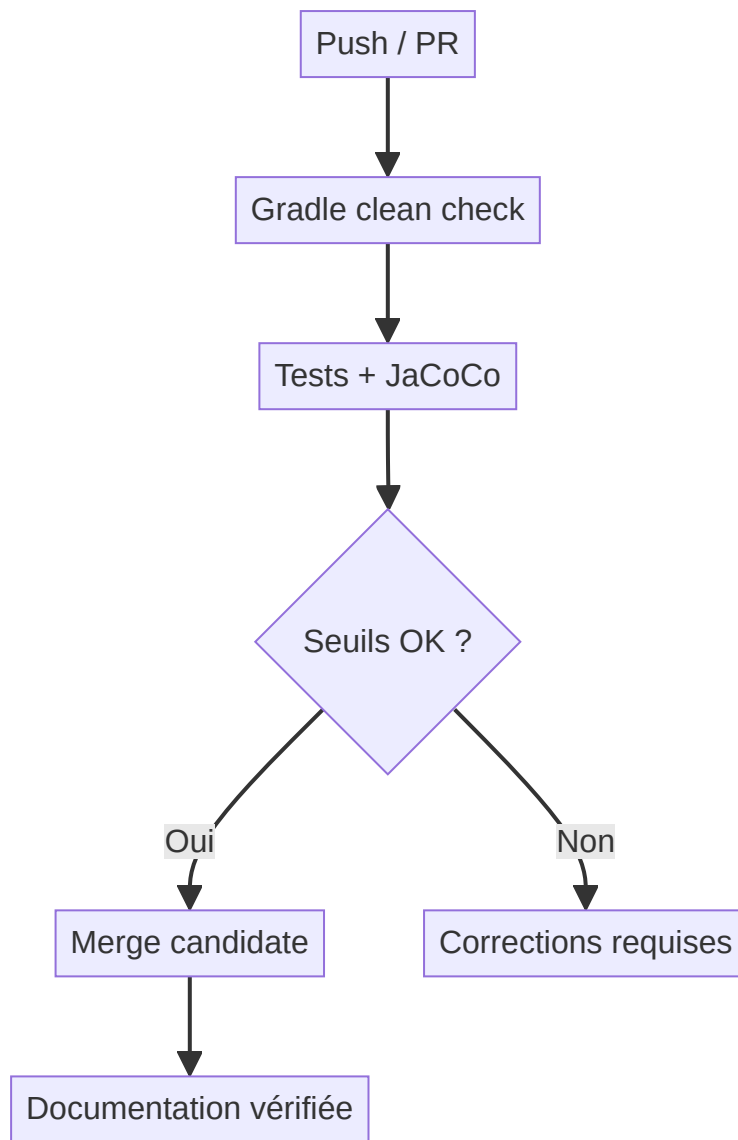
GreenDesk Sessions

GreenDesk

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
org.example		86%		28%	23 41	14 110	2 20	0 2
org.example.controllers.greenhouse		82%		66%	6 20	7 39	0 11	0 2
org.example.entities.ecosystem		93%		75%	17 49	10 101	2 17	0 2
org.example.controllers.forest		89%		n/a	0 15	9 61	0 15	0 2
org.example.entities.forest		95%		78%	12 54	6 70	3 33	0 2
org.example.entities.ecosystem.diseases		93%		70%	6 29	3 51	3 24	0 3
org.example.entities.species		94%		80%	2 31	1 56	1 26	0 1
org.example.entities.plant		99%		100%	1 60	1 125	1 49	1 4
org.example.entities.environment		99%		94%	1 63	1 120	0 52	0 4
org.example.entities		100%		n/a	0 35	0 64	0 35	0 4
org.example.entities.effect		100%		n/a	0 27	0 63	0 27	0 1
org.example.controllers.species		100%		n/a	0 7	0 20	0 7	0 1
org.example.controllers.stimulus		100%		n/a	0 2	0 5	0 2	0 1
org.example.controllers.environment		100%		n/a	0 2	0 3	0 2	0 1
org.example.controllers		100%		n/a	0 2	0 2	0 2	0 1
Total	219 of 3,657	94%	67 of 227	70%	68 437	52 890	12 322	1 31

Created with JaCoCo 0.8.12.202403310830

6.2 CI/CD (Documentation complète + CI)

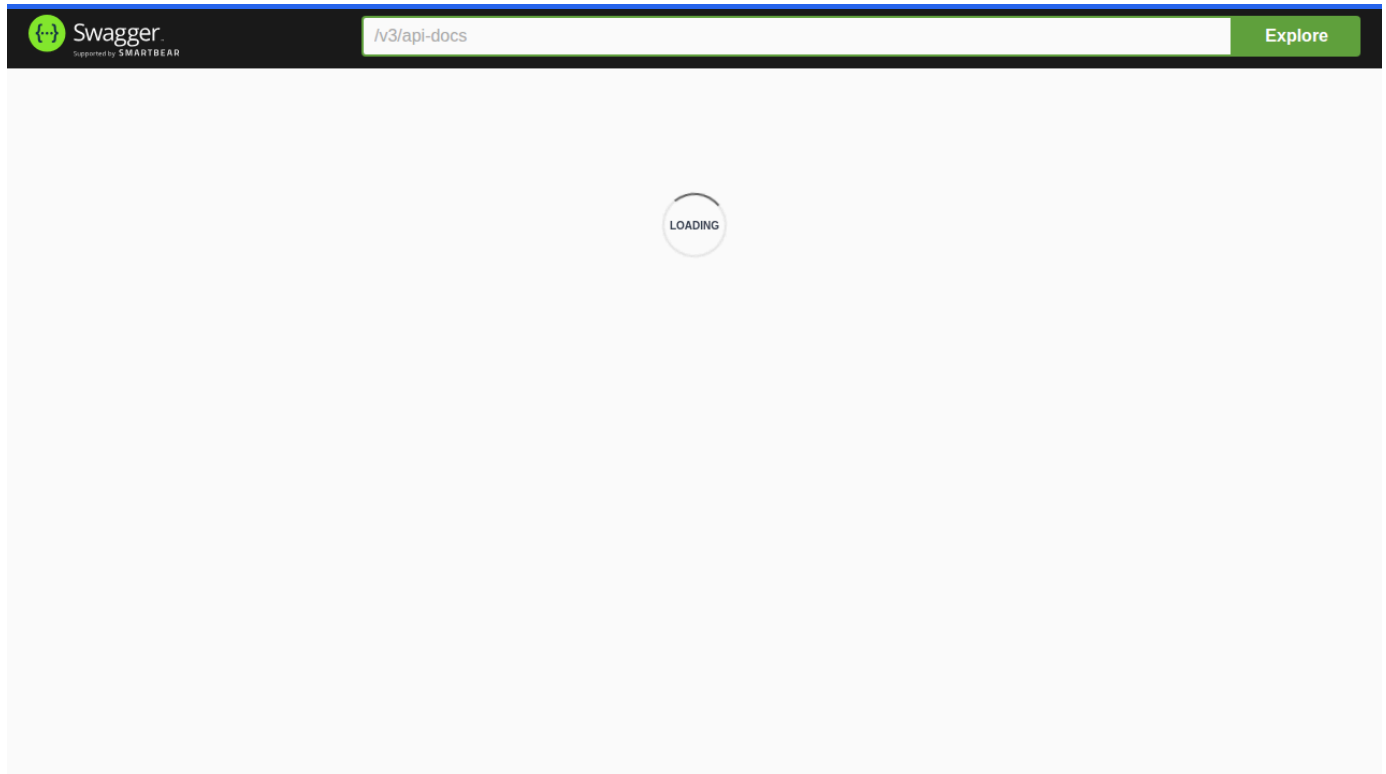


Commandes standard :

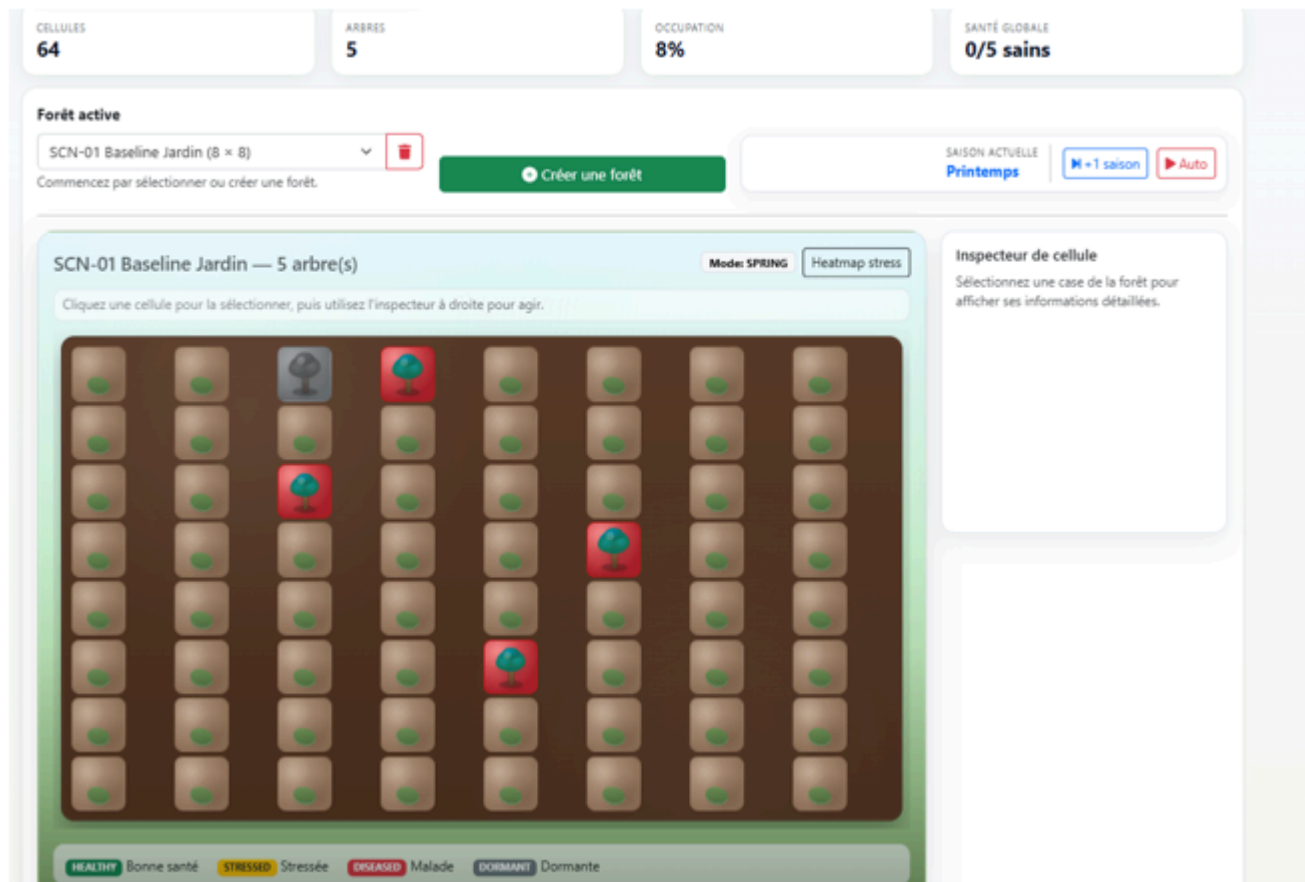
```
./gradlew test
./gradlew clean check
./gradlew test jacocoTestReport
```

bash

6.3 Captures qualité



Capture test forêt



7. Guide d'Installation & Déploiement

7.1 Prérequis

- Java 21
- Docker (optionnel mais recommandé)
- Port 8080 disponible

7.2 Exécution locale

```
./gradlew clean bootRun
```

bash

- App : <http://localhost:8080/>
- Swagger : <http://localhost:8080/swagger-ui/index.html>

7.3 Exécution Docker

```
docker compose up -d --build
```

bash

- App : <http://localhost:8080>
- Mongo Express : <http://localhost:8081>

7.4 Vérifications rapides

```
curl -s http://localhost:8080/api/species  
curl -s http://localhost:8080/api/forests  
curl -s http://localhost:8080/api/greenhouse/overview
```

bash

7.5 Dépannage

- API inaccessible : vérifier port 8080 et logs applicatifs.
- Mongo indisponible : vérifier URI, credentials, service DB.
- Tests en échec : ouvrir le rapport tests et corriger par lot.

8. Annexe API REST

8.1 Base URL

`http://localhost:8080`

8.2 Endpoints principaux

Domaine	Méthode	Endpoint
Espèces	GET	<code>/api/species</code>
Espèces	POST	<code>/api/species</code>
Plantes	POST	<code>/api/plants/create</code>
Plantes	GET	<code>/api/plants/{id}/status</code>
Forêts	POST	<code>/api/forests</code>
Forêts	POST	<code>/api/forests/{forestId}/plants</code>
Saisons	POST	<code>/api/forests/{id}/season-cycle/advance</code>
Effets	POST	<code>/api/plants/{plantId}/effects/{effectId}</code>
Stimulus	POST	<code>/api/stimuli</code>
Alertes	GET	<code>/plants/{plantId}/alerts</code>
Alertes	POST	<code>/alerts/{alertId}/ack</code>
Écosystème	POST	<code>/api/ecosystem/simulate/{n}</code>
Greenhouse	GET	<code>/api/greenhouse/overview</code>
Greenhouse	GET	<code>/api/greenhouse/roi</code>
Greenhouse	POST	<code>/api/greenhouse/sensor-stream/tick</code>

8.3 Exemples payload

Créer espèce

```
{
  "name": "Basilic",
  "optimalWaterNeeds": 45,
  "optimalTemperature": 24,
  "optimalHumidity": 55,
  "optimalLuxNeeds": 280,
  "baseGrowthRate": 1.1,
  "seedProductionRate": 0.9
}
```

json

Créer forêt

```
{
  "name": "Zone-Nord",
  "width": 8,
  "height": 8
}
```

json

Tick Greenhouse

```
{
  "forestId": "<FOREST_ID>",
  "profile": "NORMAL"
}
```

json

8.4 Format d'erreur structuré

```
{
  "error": "message lisible",
  "endpoint": "/api/greenhouse/...",
  "timestamp": "2026-02-26T12:34:56"
}
```

json